

Recurrences

mahmed.cse@mnit.ac.in

Recurrences

- A natural way to define the problem in terms of the problem itself.
- Recursion is useful for problems that can be represented by a **simpler version** of the same problem.
- Example: Factorial Function

$$6! = 6 * 5 * 4 * 3 * 2 * 1$$

This can also be written as:

$$6! = 6 * 5!$$

Recurrences

In general, we can express the factorial function as follows:

$$n! = n * (n-1) !$$

Check the correctness if any required !

The factorial function is only defined for *positive* integers. So we should be a bit more precise:

$$n! = 1 \quad (\text{if } n \text{ is equal to } 1)$$

$$n! = n * (n-1) ! \quad (\text{if } n \text{ is larger than } 1)$$

Recurrences and Running Time

- The generalized equation in terms of an equation or inequality to describes a function in terms of its value on smaller inputs.

$$T(n) = T(n-1) + n$$

- Recurrences arise when an algorithm contains recursive calls to itself
- What is the actual running time of the algorithm?
- Need to solve the recurrence
 - Find an explicit formula of the expression
 - Bound the recurrence by an expression that involves n

Ex. 1: The Handshake Problem

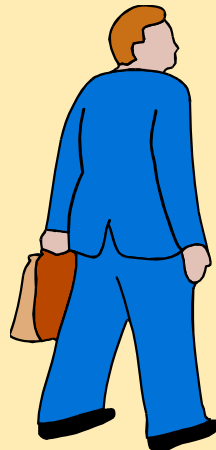
There are n people in a room. If each person shakes hands once with every other person. What is the total number $h(n)$ of handshakes?

$$h(n) = h(n-1) + n-1$$

$$h(4) = h(3) + 3$$

$$h(3) = h(2) + 2$$

$$h(2) = 1$$



$$h(n): \text{Sum of integer from } 1 \text{ to } n-1 = n(n-1) / 2$$

Recursion

We have to pay a price for recursion:

- calling a function **consumes more time and memory** than adjusting a loop counter.
- high performance applications (graphic action games, simulations of nuclear explosions) hardly ever use recursion.

In less demanding applications recursion is an attractive alternative for iteration (for the right problems!)

Recursion

If we use iteration, we must be careful not to create an infinite loop by accident:

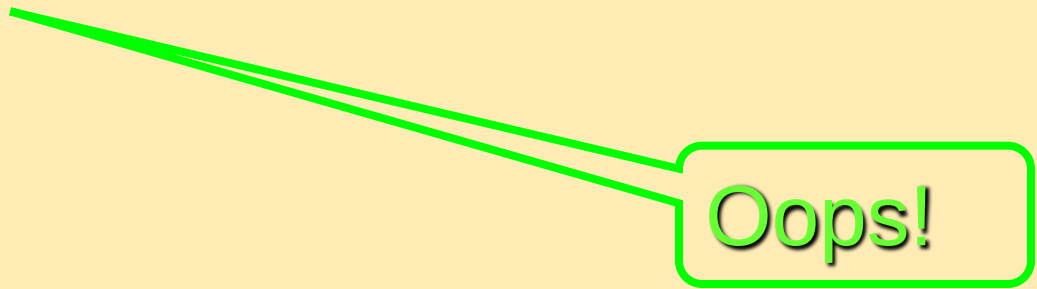
```
for(int incr=1; incr!=10;incr+=2)
```

```
...
```

```
int result = 1;  
while(result >0) {  
    ...  
    result++;  
}
```



Oops!



Oops!

Recursion

Similarly, if we use recursion we must be careful not to create an infinite chain of function calls:

```
int fac(int numb){  
    return numb * fac(numb-1);  
}
```

Oops!
No termination
condition

Or:

```
int fac(int numb){  
    if (numb<=1)  
        return 1;  
    else  
        return numb * fac(numb+1);  
}
```

Oops!

Re-occurring Sub-instances

- In divide-&-conquer algorithms such as MergeSort, an instance is typically partitioned into “non-overlapping” sub-instances (e.g., two disjoint sub-arrays). That results in the following property:
 - For any two (sub) sub-instances, either one is a sub-instance of the other (a descendant in the recursion tree), or the two are disjoint & independent from each other.
 - We never encounter the same sub-instance again during the entire recursion.
- If the sub-instances “overlap”, further down the recursion tree we may encounter repeated (sub) sub-instances that are in their “common intersection”.
 - Example 1: The Fibonacci Recursion tree (see next slide).
 - Example 2: Overlapping sub-arrays as sub-instances.

Fibonacci Recursion Tree

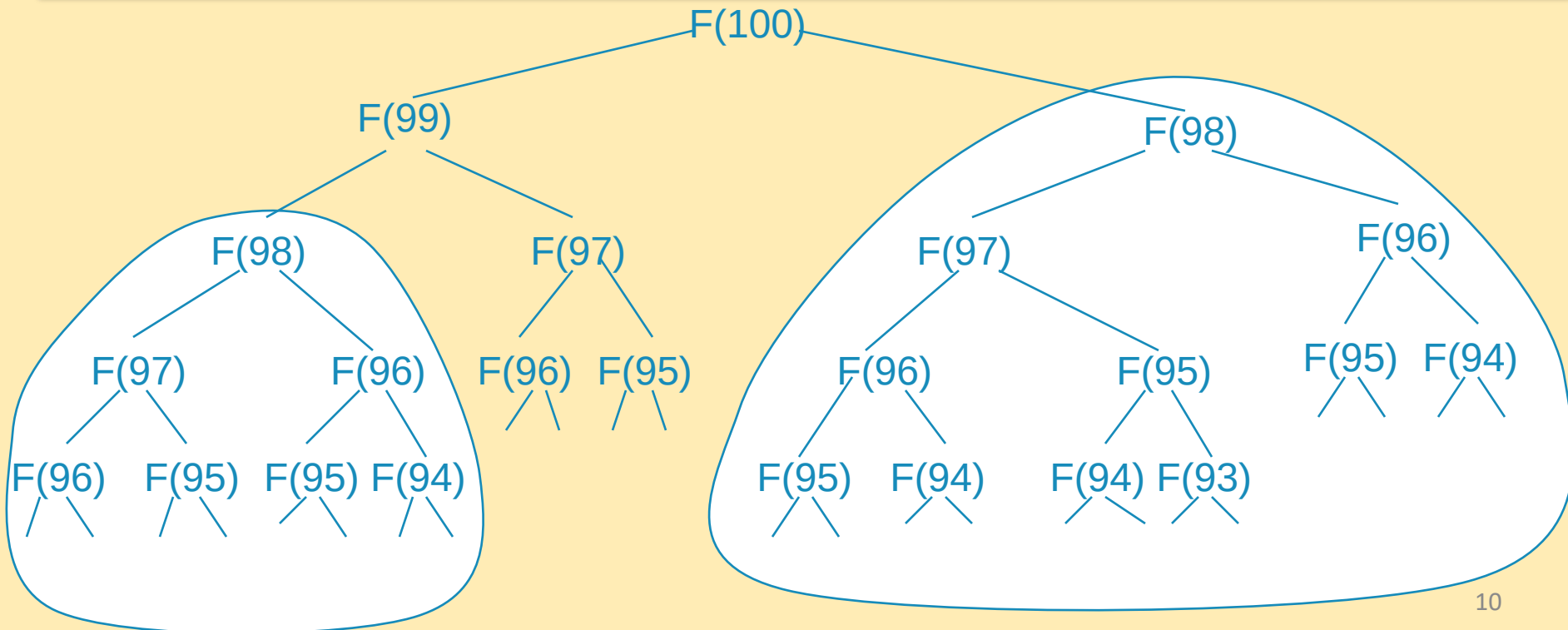
```
Algorithm  $F(n)$   
  if  $n \in \{0,1\}$  then return  $n$   
  return  $F(n-1) + F(n-2)$   
end
```

$$T(n) = \begin{cases} T(n-1) + T(n-2) + \Theta(1) & n \geq 2 \\ \Theta(1) & n = 0,1 \end{cases}$$
$$\Rightarrow T(n) = \Theta(F_n) = \Theta(\varphi^n).$$

But only $n+1$ distinct sub-instances are ever called:

$F(0), F(1), \dots, F(n-2), F(n-1), F(n)$.

Why is it taking time exponential in n ?



Pruning by Memoization

Algorithm **Fib(n)**

```
for t ← 0 .. 1 do Memo[t] ← t  
for t ← 2 .. n do Memo[t] ← null  
return F(n)
```

§ initialize Memo[0..n] table

end

Function **F(n)**

```
if Memo[n] = null then Memo[n] ← F(n-1) + F(n-2)  
return Memo[n]
```

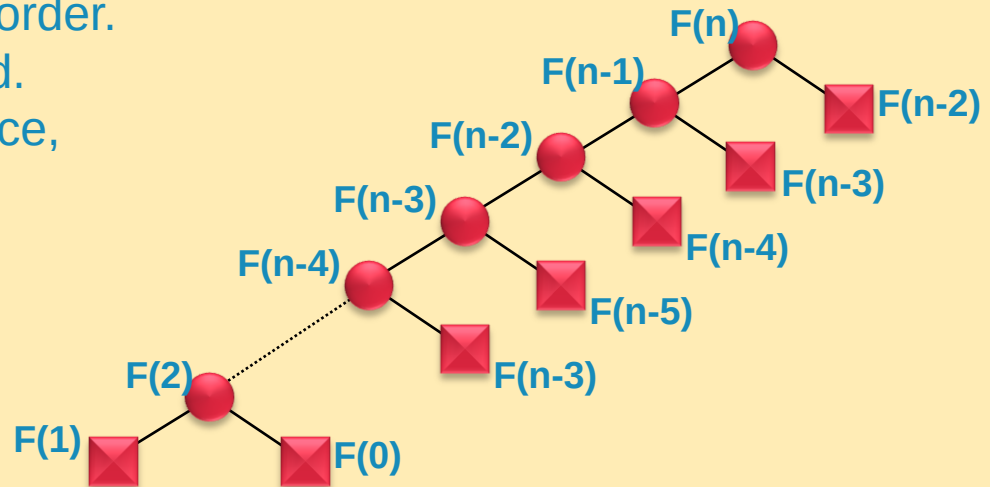
§ recursion structure kept
§ memo feature added

end

Recursion tree evaluated in post-order.
Now all right sub-trees are pruned.
Instead of re-solving a sub-instance,
fetch its solution from
the memo table Memo[0..n].

$$T(n) = \Theta(n).$$

Time-space trade off.



Memoization: Recursive vs Iterative

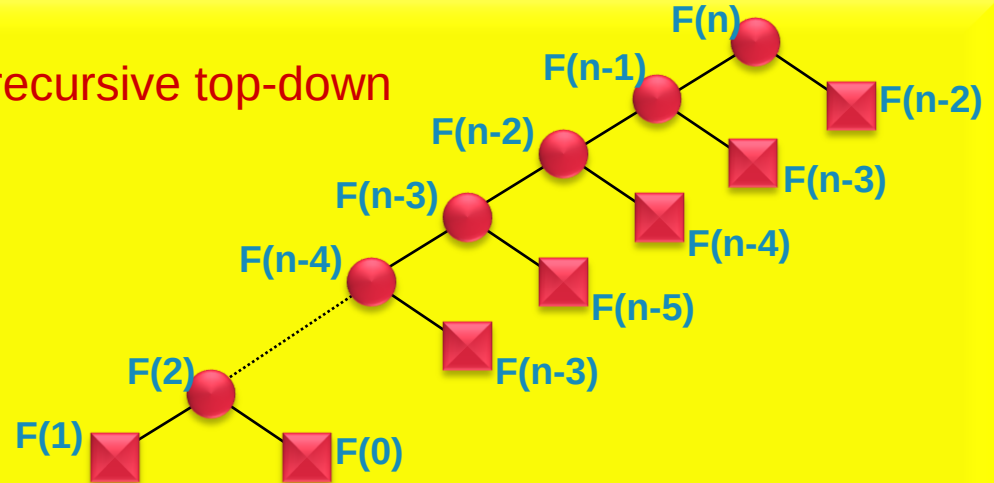
Algorithm **Fib(n)**

```
for t ← 0 .. 1 do Memo[t] ← t
for t ← 2 .. n do Memo[t] ← null
return F(n)
end
```

Function **F(n)**

```
if Memo[n] = null
  then Memo[n] ← F(n-1) + F(n-2)
return Memo[n]
end
```

§ recursive top-down



Algorithm **Fib(n)**

```
for t ← 0 .. 1 do Memo[t] ← t
for t ← 2 .. n do Memo[t] ← Memo[t-1] + Memo[t-2]
return Memo[n]
end
```

§ iterative bottom-up (from smallest to largest)

* Compact memorisation

Example: Recurrences

- $T(n) = T(n-1) + n$ $\Theta(n^2)$
 - Recursive algorithm that loops through the input to eliminate one item
- $T(n) = T(n/2) + c$ $\Theta(\lg n)$
 - Recursive algorithm that halves the input in one step
- $T(n) = T(n/2) + n$ $\Theta(n)$
 - Recursive algorithm that halves the input but must examine every item in the input
- $T(n) = 2T(n/2) + 1$ $\Theta(n)$
 - Recursive algorithm that splits the input into 2 halves and does a constant amount of other work

Recurrent Algorithms

BINARY-SEARCH

- for an **ordered array** A , finds if x is in the array $A[\text{lo} \dots \text{hi}]$

Alg.: BINARY-SEARCH ($A, \text{lo}, \text{hi}, x$)

if ($\text{lo} > \text{hi}$)

return FALSE

$\text{mid} \leftarrow \lfloor (\text{lo} + \text{hi}) / 2 \rfloor$

if $x = A[\text{mid}]$

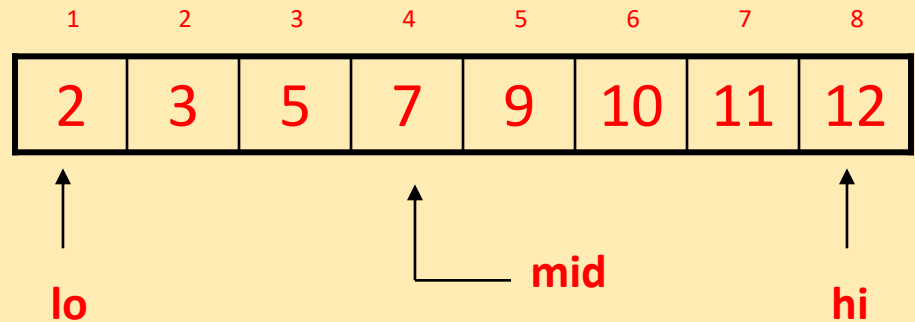
return TRUE

if ($x < A[\text{mid}]$)

 BINARY-SEARCH ($A, \text{lo}, \text{mid}-1, x$)

if ($x > A[\text{mid}]$)

 BINARY-SEARCH ($A, \text{mid}+1, \text{hi}, x$)



Example

- $A[8] = \{1, 2, 3, 4, 5, 7, 9, 11\}$

– $lo = 1$ $hi = 8$ $x = 7$

1	2	3	4	5	6	7	8
1	2	3	4	5	7	9	11

$mid = 4, lo = 5, hi = 8$

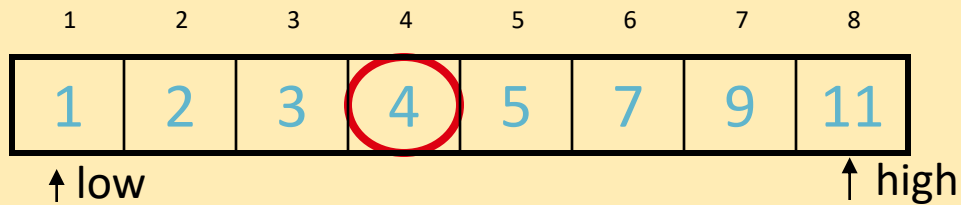
				5	6	7	8
1	2	3	4	5	7	9	11

$mid = 6, A[mid] = x$
Found!

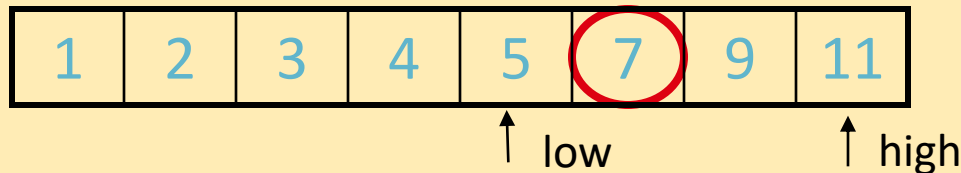
Another Example

- $A[8] = \{1, 2, 3, 4, 5, 7, 9, 11\}$

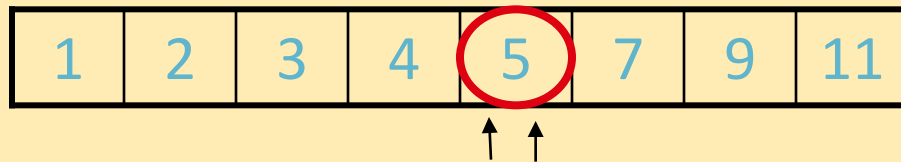
– lo = 1 hi = 8 **x = 6**



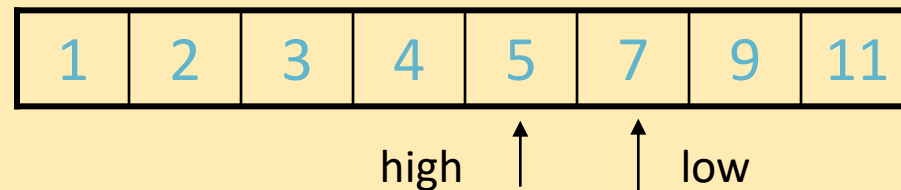
mid = 4, lo = 5, hi = 8



mid = 6, A[6] = 7, lo = 5, hi = 5



mid = 5, A[5] = 5, lo = 6, hi = 5
NOT FOUND!



Analysis of BINARY-SEARCH

Alg.: BINARY-SEARCH (A, lo, hi, x)

if (lo > hi)



constant time: c_1

return FALSE

mid $\leftarrow \lfloor (lo+hi)/2 \rfloor$



constant time: c_2

if x = A[mid]



constant time: c_3

return TRUE

if (x < A[mid])

BINARY-SEARCH (A, lo, mid-1, x)

← same problem of size $n/2$

if (x > A[mid])

BINARY-SEARCH (A, mid+1, hi, x)

← same problem of size $n/2$

- $T(n) = c + T(n/2)$

- $T(n)$ – running time for an array of size n

Methods for Solving Recurrences

- Iteration method
- Substitution method
- Recursion tree method
- Master method

The Iteration Method

- Convert the recurrence into a summation and try to bound it using known series
 - Iterate the recurrence until the initial condition is reached.
 - Use back-substitution to express the recurrence in terms of n and the initial (boundary) condition.

The Iteration Method

$$T(n) = c + T(n/2)$$

$$\begin{aligned} T(n) &= c + T(n/2) \\ &= c + c + T(n/4) \\ &= c + c + c + T(n/8) \end{aligned}$$

Assume $n = 2^k$

$$T(n) = \underbrace{c + c + \dots + c}_{k \text{ times}} + T(1)$$

$$\begin{aligned} &= c \lg n + T(1) \\ &= \Theta(\lg n) \end{aligned}$$

$$T(n/2) = c + T(n/4)$$

$$T(n/4) = c + T(n/8)$$

Iteration Method – Example

$$T(n) = n + 2T(n/2) \quad \text{Assume: } n = 2^k$$

$$T(n/2) = n/2 + 2T(n/4)$$

$$\begin{aligned} T(n) &= n + 2T(n/2) \\ &= n + 2(n/2 + 2T(n/4)) \\ &= n + n + 4T(n/4) \\ &= n + n + 4(n/4 + 2T(n/8)) \\ &= n + n + n + 8T(n/8) \\ \dots &= in + 2^iT(n/2^i) \\ &= kn + 2^kT(1) \\ &= n \lg n + nT(1) = \Theta(n \lg n) \end{aligned}$$

The substitution method

1. Guess a solution
2. Use induction to prove that the solution works

Substitution method

- Guess a solution
 - $T(n) = O(g(n))$
 - Induction goal: **apply the definition of the asymptotic notation**
 - $T(n) \leq d g(n)$, for some $d > 0$ and $n \geq n_0$ (strong induction)
 - Induction hypothesis: $T(k) \leq d g(k)$ for all $k < n$
- Prove the induction goal
 - Use the **induction hypothesis** to **find some values of the constants d and n_0** for which the **induction goal** holds

Example: Binary Search

$$T(n) = c + T(n/2)$$

- Guess: $T(n) = O(\lg n)$
 - Induction goal: $T(n) \leq d \lg n$, for some d and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq d \lg(n/2)$

- Proof of induction goal:

$$\begin{aligned} T(n) &= T(n/2) + c \leq d \lg(n/2) + c \\ &= d \lg n - d + c \leq d \lg n \end{aligned}$$

$$\text{if: } -d + c \leq 0, d \geq c$$

- Base case?

Example 2

$$T(n) = T(n-1) + n$$

- Guess: $T(n) = O(n^2)$
 - Induction goal: $T(n) \leq c n^2$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(n-1) \leq c(n-1)^2$ for all $k < n$

- Proof of induction goal:

$$T(n) = T(n-1) + n \leq c(n-1)^2 + n$$

$$= cn^2 - (2cn - c - n) \leq cn^2$$

$$\text{if: } 2cn - c - n \geq 0 \Leftrightarrow c \geq n/(2n-1) \Leftrightarrow c \geq 1/(2 - 1/n)$$

- For $n \geq 1 \Rightarrow 2 - 1/n \geq 1 \Rightarrow$ any $c \geq 1$ will work

Example 3

$$T(n) = 2T(n/2) + n$$

- Guess: $T(n) = O(n \lg n)$
 - Induction goal: $T(n) \leq cn \lg n$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq cn/2 \lg(n/2)$
- Proof of induction goal:
$$\begin{aligned} T(n) &= 2T(n/2) + n \leq 2c (n/2) \lg(n/2) + n \\ &= cn \lg n - cn + n \leq cn \lg n \end{aligned}$$

if: $-cn + n \leq 0 \Rightarrow c \geq 1$
- Base case?

Changing variables

$$T(n) = 2T(\sqrt{n}) + \lg n$$

– Rename: $m = \lg n \Rightarrow n = 2^m$

$$T(2^m) = 2T(2^{m/2}) + m$$

– Rename: $S(m) = T(2^m)$

$$S(m) = 2S(m/2) + m \Rightarrow S(m) = O(m \lg m)$$

(demonstrated before)

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$$

Idea: transform the recurrence to one that you have seen before

The recursion-tree method

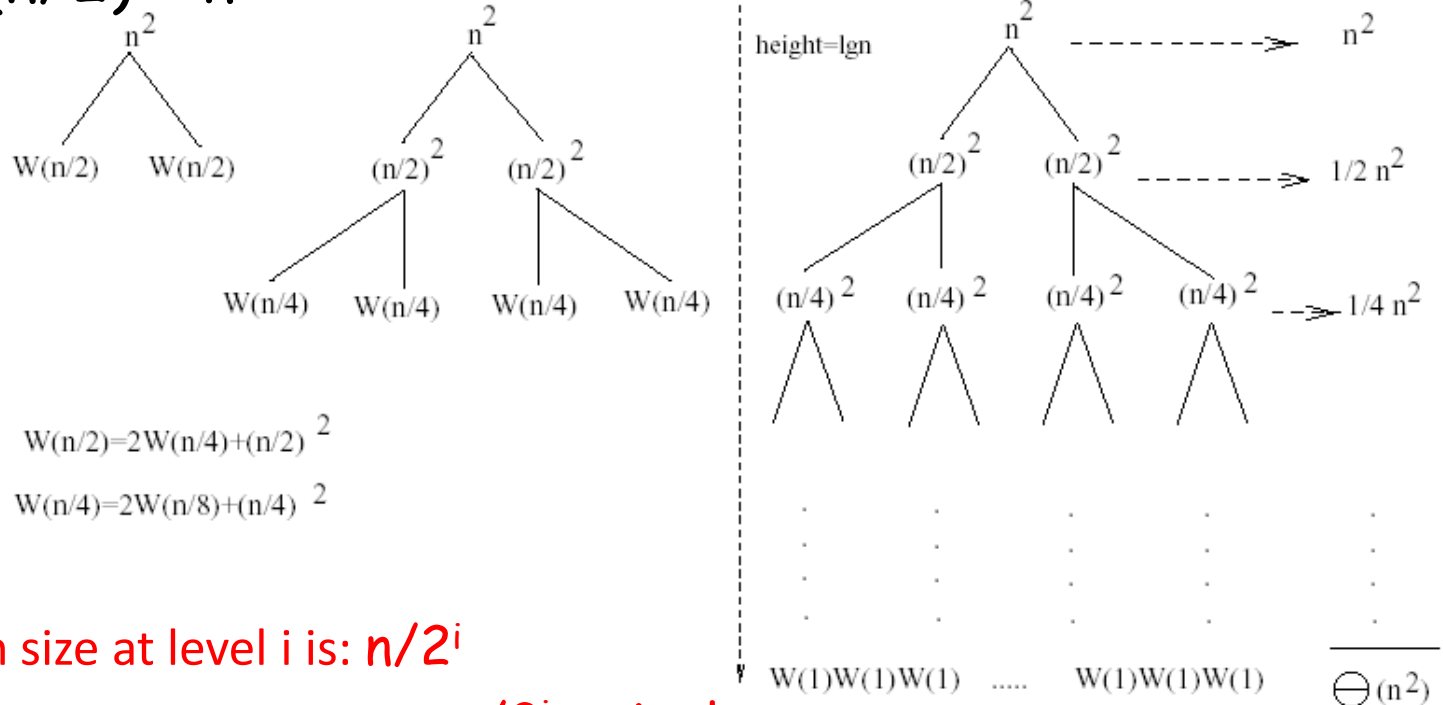
Convert the recurrence into a tree:

- Each node represents the cost incurred at various levels of recursion
- Sum up the costs of all levels

Used to “guess” a solution for the recurrence

Example 1

$$W(n) = 2W(n/2) + n^2$$



- Subproblem size at level i is: $n/2^i$
- Subproblem size hits 1 when $1 = n/2^i \Rightarrow i = \lg n$
- Cost of the problem at level $i = (n/2^i)^2$ No. of nodes at level $i = 2^i$

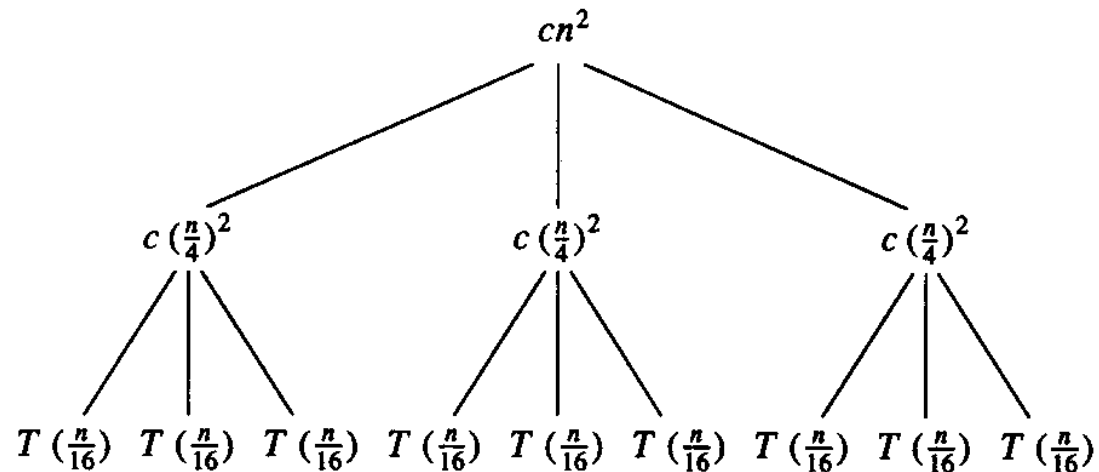
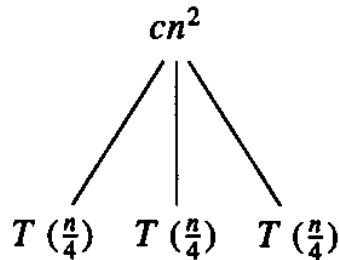
• Total cost:

$$W(n) = \sum_{i=0}^{\lg n - 1} \frac{n^2}{2^i} + 2^{\lg n} W(1) = n^2 \sum_{i=0}^{\lg n - 1} \left(\frac{1}{2}\right)^i + n \leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i + O(n) = n^2 \frac{1}{1 - 1/2} + O(n) = 2n^2$$

$$\Rightarrow W(n) = O(n^2)$$

Example 2

E.g.: $T(n) = 3T(n/4) + cn^2$



- Subproblem size at level i is: $n/4^i$
- Subproblem size hits 1 when $1 = n/4^i \Rightarrow i = \log_4 n$
- Cost of a node at level $i = c(n/4^i)^2$
- Number of nodes at level $i = 3^i \Rightarrow$ last level has $3^{\log_4 n} = n^{\log_4 3}$ nodes
- Total cost:

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \leq \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) = \frac{1}{1 - \frac{3}{16}} cn^2 + \Theta(n^{\log_4 3}) = O(n^2)$$

$\Rightarrow T(n) = O(n^2)$

Example 2 - Substitution

$$T(n) = 3T(n/4) + cn^2$$

- Guess: $T(n) = O(n^2)$
 - Induction goal: $T(n) \leq dn^2$, for some d and $n \geq n_0$
 - Induction hypothesis: $T(n/4) \leq d(n/4)^2$
- Proof of induction goal:

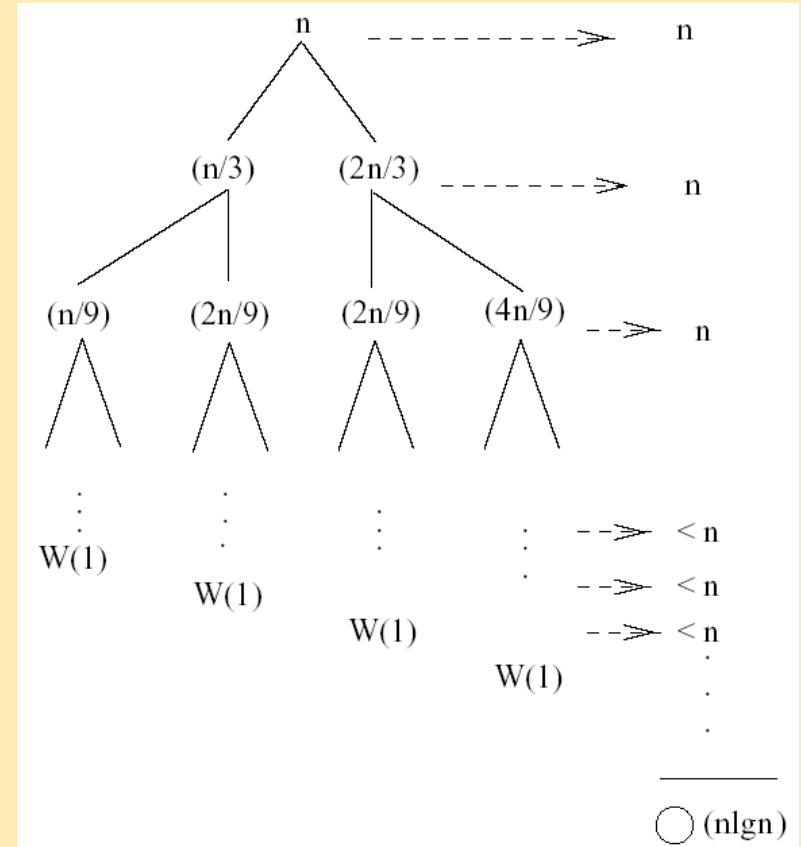
$$\begin{aligned} T(n) &= 3T(n/4) + cn^2 \\ &\leq 3d(n/4)^2 + cn^2 \\ &= (3/16)d n^2 + cn^2 \\ &\leq d n^2 \quad \text{if: } d \geq (16/13)c \end{aligned}$$

- Therefore: $T(n) = O(n^2)$

Example 3 (simpler proof)

$$W(n) = W(n/3) + W(2n/3) + n$$

- The longest path from the root to a leaf is: n
 $\rightarrow (2/3)n \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow 1$
- Subproblem size hits 1 when $1 = (2/3)^i n \Leftrightarrow i = \log_{3/2} n$
- Cost of the problem at level $i = n$
- Total cost:



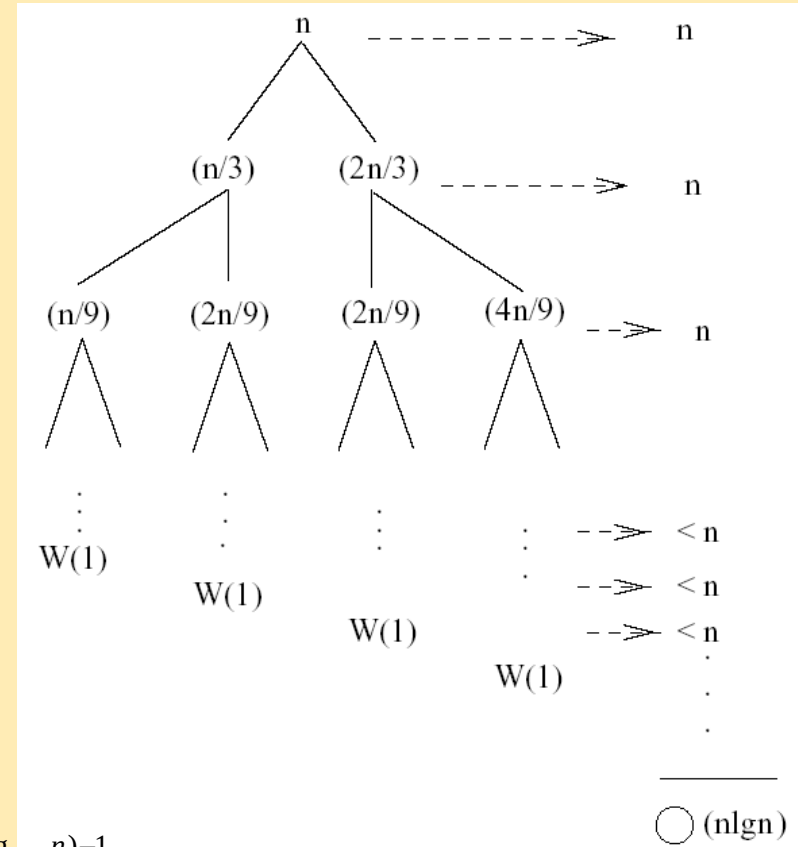
$$W(n) < n + n + \dots = n(\log_{3/2} n) = n \frac{\lg n}{\lg \frac{3}{2}} = O(n \lg n)$$

$$\Rightarrow W(n) = O(n \lg n)$$

Example 3

$$W(n) = W(n/3) + W(2n/3) + n$$

- The longest path from the root to a leaf is:
 $\rightarrow (2/3)n \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow 1$
- Subproblem size hits 1 when $1 = (2/3)^i n \Leftrightarrow i = \log_{3/2} n$
- Cost of the problem at level $i = n$
- Total cost:



$$W(n) < n + n + \dots = \sum_{i=0}^{(\log_{3/2} n) - 1} n + 2^{(\log_{3/2} n)} W(1) <$$

$$< n \sum_{i=0}^{\log_{3/2} n} 1 + n^{\log_{3/2} 2} = n \log_{3/2} n + O(n) = n \frac{\lg n}{\lg 3/2} + O(n) = \frac{1}{\lg 3/2} n \lg n + O(n)$$

$$\Rightarrow W(n) = O(n \lg n)$$

Example 3 - Substitution

$$W(n) = W(n/3) + W(2n/3) + O(n)$$

- Guess: $W(n) = O(n \lg n)$
 - Induction goal: $W(n) \leq d n \lg n$, for some d and $n \geq n_0$
 - Induction hypothesis: $W(k) \leq d k \lg k$ for any $K < n$ ($n/3, 2n/3$)
- Proof of induction goal:

Try it out as an exercise!!

- $T(n) = O(n \lg n)$

Master's method

- “Cookbook” for solving recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Idea: compare $f(n)$ with $n^{\log_b a}$

- $f(n)$ is asymptotically smaller or larger than $n^{\log_b a}$ by a polynomial factor n^ϵ
- $f(n)$ is asymptotically equal with $n^{\log_b a}$

Master's method

- “Cookbook” for solving recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Case 1: if $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then: $T(n) = \Theta(n^{\log_b a})$

Case 2: if $f(n) = \Theta(n^{\log_b a})$, then: $T(n) = \Theta(n^{\log_b a} \lg n)$

Case 3: if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and if

$af(n/b) \leq cf(n)$ for some $c < 1$ and all sufficiently large n , then:



$$T(n) = \Theta(f(n))$$

regularity condition

Examples

$$T(n) = 2T(n/2) + n$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare $n^{\log_2 2}$ with $f(n) = n$

$$\Rightarrow f(n) = \Theta(n) \Rightarrow \text{Case 2}$$

$$\Rightarrow T(n) = \Theta(n \lg n)$$

Examples

$$T(n) = 2T(n/2) + n^2$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare n with $f(n) = n^2$

$\Rightarrow f(n) = \Omega(n^{1+\varepsilon})$ Case 3 $\Rightarrow$ verify regularity cond.

$$a f(n/b) \leq c f(n)$$

$$\Leftrightarrow 2 n^2/4 \leq c n^2 \Rightarrow c = \frac{1}{2} \text{ is a solution } (c < 1)$$

$$\Rightarrow T(n) = \Theta(n^2)$$

Examples (cont.)

$$T(n) = 2T(n/2) + \sqrt{n}$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare n with $f(n) = n^{1/2}$

$$\Rightarrow f(n) = O(n^{1-\varepsilon}) \quad \text{Case 1}$$

$$\Rightarrow T(n) = \Theta(n)$$

Examples

$$T(n) = 3T(n/4) + n \lg n$$

$$a = 3, b = 4, \log_4 3 = 0.793$$

Compare $n^{0.793}$ with $f(n) = n \lg n$

$$f(n) = \Omega(n^{\log_4 3 + \epsilon}) \text{ Case 3}$$

Check regularity condition:

$$3 * (n/4) \lg(n/4) \leq (3/4) n \lg n = c * f(n),$$

$$c = 3/4$$

Examples

$$T(n) = 2T(n/2) + n \lg n$$

$$a = 2, b = 2, \log_2 2 = 1$$

- Compare n with $f(n) = n \lg n$
 - seems like case 3 should apply
- $f(n)$ must be polynomially larger by a factor of n^ϵ
- In this case it is only larger by a factor of $\lg n$

Readings